

README

PDF를 업로드하거나 서버 경로로 지정해 목차 생성을 수행하는 FastAPI 서버입니다.

처리는 비동기 job 방식입니다. 요청을 보내면 즉시 `job_id` 를 반환하고, 클라이언트는 상태 조회 API로 진행 상태와 결과 파일을 확인합니다.

처리 단계

```
PDF
-> PDF layout text 파싱
-> OpenAI로 layout rule JSON 생성
-> Gemma/vLLM으로 최종 TOC JSON 생성
```

단계별 API는 분리되어 있습니다.

```
/jobs/layout/upload  layout JSON만 생성
/jobs/gemma/upload   기존 layout JSON으로 최종 TOC JSON 생성
/jobs/toc/upload     layout 생성 + 최종 TOC 생성
```

구성

Contents_ai_api/	
├ app.py	FastAPI 앱 생성
├ routes.py	API 엔드포인트
├ jobs.py	job 생성 및 백그라운드 실행
├ storage.py	업로드 파일과 job 상태 저장
├ full_toc_core.py	PDF 파싱, OpenAI layout, Gemma 실행
├ full_toc_v4.py	서버 실행 진입점
├ requirements.txt	
├ smoke_test.py	외부 AI 서버 없이 API/job 흐름 검증
└ PROCESS.md	상세 프로세스 문서

설치

```
cd /path/to/Contents_ai_api
python3 -m venv .venv
```

```
source .venv/bin/activate
pip install -r requirements.txt
```

이미 서버에 공용 가상환경이 있으면 해당 환경을 사용해도 됩니다.

API Key 설정

layout 생성에는 프로젝트 루트의 `.env` 파일에 아래 값이 있으면 됩니다.

```
OPENAI_API_KEY=sk-...
```

`.env` 를 새로 만들거나 수정한 뒤에는 FastAPI 서버를 재시작해야 합니다.

Gemma/vLLM 서버

`/jobs/gemma/upload` 또는 `/jobs/toc/upload` 는 Gemma/vLLM OpenAI-compatible 서버가 필요합니다.

예시:

```
CUDA_VISIBLE_DEVICES=4,5,6,7 VLLM_USE_FLASHINFER_SAMPLER=0
\
python -m vllm.entrypoints.openai.api_server \
  --model google/gemma-4-31B-it \
  --served-model-name google/gemma-4-31B-it \
  --host 127.0.0.1 \
  --port 8000 \
  --tensor-parallel-size 4 \
  --max-model-len 262144 \
  --gpu-memory-utilization 0.9 \
  --trust-remote-code
```

기본 Gemma API base URL은 `http://127.0.0.1:8000/v1` 입니다.

FastAPI 서버 실행

로컬 테스트:

```
python full_toc_v4.py --host 127.0.0.1 --port 8080
```

외부에서 접속해야 하는 서버:

```
python full_toc_v4.py --host 0.0.0.0 --port 8080
```

상태 확인:

```
curl <http://127.0.0.1:8080/health>
```

정상 응답:

```
{"ok":true,"service":"full_toc_v4","framework":"fastapi"}
```

브라우저 문서:

```
<http://127.0.0.1:8080/docs>
```

파일 업로드 예시

상대경로는 FastAPI 서버 기준이 아니라 `curl` 을 실행하는 터미널의 현재 위치 기준입니다.

파일 확인:

```
ls -lh ../Contents_ai_test/data/input/1_2.pdf
```

layout만 생성:

```
curl -X POST <http://127.0.0.1:8080/jobs/layout/upload> \
-F 'file=@../Contents_ai_test/data/input/1_2.pdf' \
-F 'config={"parse_mode":"headings"}'
```

전체 TOC 생성:

```
curl -X POST <http://127.0.0.1:8080/jobs/toc/upload> \
-F 'file=@../Contents_ai_test/data/input/1_2.pdf' \
-F 'config={"parse_mode":"headings"}'
```

응답 예:

```
{
  "job_id": "20260727_111930_f30d8869",
  "status": "queued",
  "stage": "all",
  "status_url": "/jobs/20260727_111930_f30d8869",
  "result_url": "/jobs/20260727_111930_f30d8869/result"
}
```

Job 확인

상태 확인:

```
curl <http://127.0.0.1:8080/jobs/JOB_ID>
```

결과 메타데이터 확인:

```
curl <http://127.0.0.1:8080/jobs/JOB_ID/result>
```

실제 결과 파일 내용 확인:

```
curl <http://127.0.0.1:8080/jobs/JOB_ID/files/result>
```

파일로 저장:

```
curl -o toc_result.json <http://127.0.0.1:8080/jobs/JOB_ID/
files/result>
python -m json.tool toc_result.json | less
```

`less` 에서 나가려면 `q` 를 누릅니다.

저장 위치

data/uploads/	업로드된 PDF
data/parsed/	PDF에서 추출한 layout text
data/layout/	OpenAI가 생성한 layout rule JSON
data/full_json/	Gemma/vLLM이 생성한 최종 TOC JSON
data/log/	실행 로그
data/jobs/	job 상태 JSON

이 산출물들은 보통 GitHub에 올리지 않습니다. 공유용 샘플이 필요하면 민감한 경로와 내용을 확인한 뒤 `examples/` 같은 별도 폴더로 복사해서 관리합니다.

Smoke Test

외부 OpenAI/Gemma 서버 없이 API/job/pipeline 흐름만 확인합니다.

```
python smoke_test.py
```

확인 범위:

```
FastAPI 앱 로드
/health
잘못된 job 404
업로드 validation
PDF 업로드
job 생성/상태 저장
PDF 파싱
layout/result/log 파일 생성
결과 파일 다운로드
```

실제 OpenAI API 연결, 실제 Gemma/vLLM 연결, 모델 품질은 확인하지 않습니다.

자주 나는 오류

```
OpenAI API key is empty. Set OPENAI_API_KEY or pass --openai-api-key.
```

`.env` 에 `OPENAI_API_KEY=sk-...` 를 설정하고 서버를 재시작합니다.

```
curl: (26) Failed to open/read local data from file/application
```

`file=@...` 경로가 curl 실행 위치 기준으로 존재하는지 `ls -lh` 로 확인합니다.

```
Gemma server is not reachable
```

vLLM OpenAI-compatible 서버가 `127.0.0.1:8000` 에서 실행 중인지 확인합니다.

상세 문서

전체 내부 처리 흐름은 [PROCESS.md](#)를 참고하세요.